MINOR PROJECT | MIDTERM PRESENTATION

Secure CAN Gateway using ARM TrustZone and OP-TEE

Secure CAN Ownership, Validation and Event Logging on Raspberry Pi 3

Aslam
Supervisor: Prof. Kolin Paul
Indian Institute of Technology Delhi

Can We Trust the CAN Event Throughout Its Lifecycle?

From receiving a CAN packet to making a security decision and storing the resulting event, each stage introduces a trust concern.

01 CAN MESSAGE TRUST

Can We Trust the CAN Message?

ECU A ↔ CAN ↔ ECU B

↓

ID | Counter | Data

- Communication: CAN enables ECUs to exchange control and monitoring messages.
- Security concern: The CAN frame itself does not provide cryptographic origin authentication.

TRUST ?

02 CAN PROCESSING TRUST

Can We Trust the Validation Decision?

CAN PACKET

↓

ID | Counter | Data

↓

Linux / Normal World (Process + Validate)

↓

ACCEPT / DROP

- Processing: The received CAN packet is handled and validated by Linux / Normal World software.
- Trust concern: If the processing environment is compromised, can we trust the validation result?

Can we trust the security decision?

03 LOG INTEGRITY

Can We Trust the Record Later?

VALIDATED EVENT

↓

ID | Counter | Time | Result

↓

Store in Memory (Security Log)

↓

Later / Analysis

- Storage: After validation, the CAN event is stored as a security record.
- Integrity concern: If the storage environment is compromised, can the recorded event be modified or erased?

Can we trust this record later?

THE RESEARCH GAP: Security-critical CAN events need trustworthy processing and trustworthy records, even when the Normal World may be compromised.

How Is Trustworthy CAN Handling Done Today?

Different approaches protect different parts of the system, but real trade-offs remain.

01 Software-Only Logging

Validation and logging in the same OS

CAN Packet
↓
Validate + Log (Linux)
↓
Log File

- Simple, no extra hardware
- Vulnerable if OS is compromised
- No hardware isolation

Protects: None (software-dependent)

02 External HSM / Logger

Security moved to separate hardware

CAN Packet
↓
External HSM / ECU
↓
Log

- Isolates security from main ECU
- Extra hardware, wiring and cost
- Added system complexity

Protects: Security record (extra hardware)

03 TrustZone + OP-TEE (Ours)

Secure-World-owned validation on-chip

CAN Packet
↓
Secure World (Core1)
↓
Accept / Drop

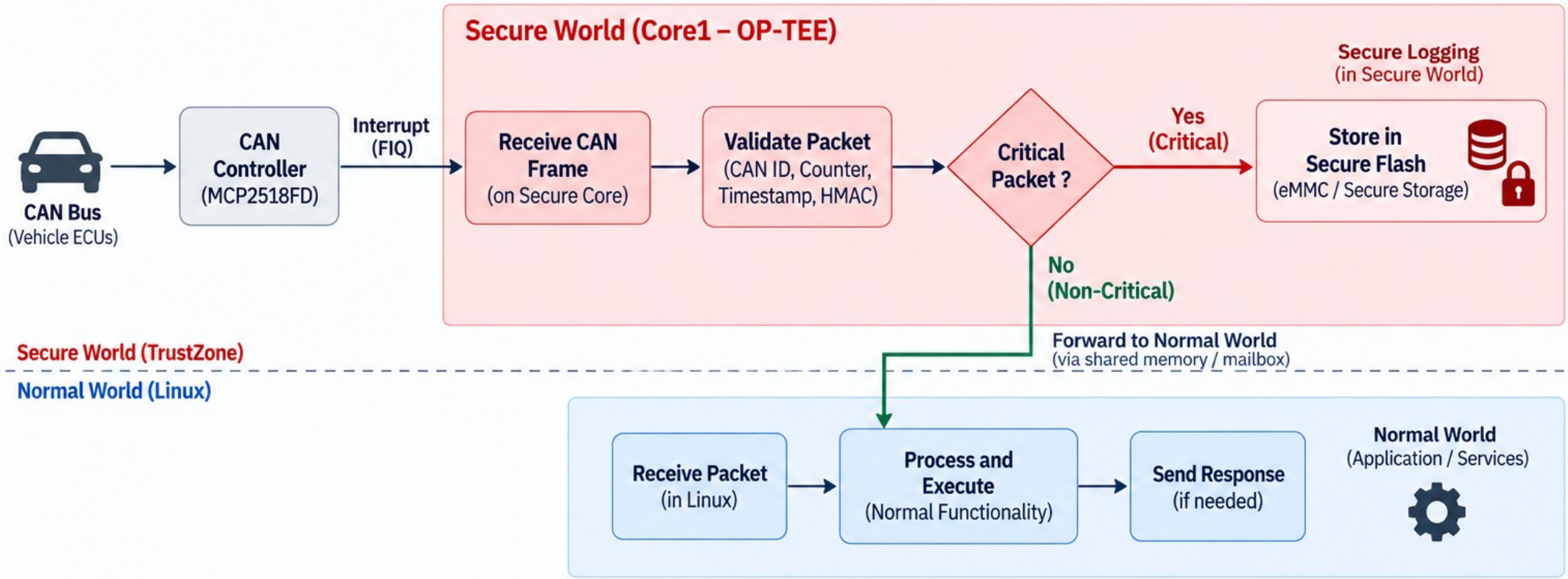
- No extra silicon - uses on-chip TrustZone
- ID / replay / timestamp checks hardware-proven
- HMAC authentication currently blocked
- Tamper-evident logging not yet implemented

Protects: Validation decision (logging planned, not built)

RESEARCH GAP: Can a single dedicated core running inside TrustZone deliver trusted, scalable CAN validation and tamper-evident logging, without dedicated security hardware, even when the Normal World is compromised?

Proposed Approach – CAN Event Handling Flow

Secure, Selective Processing using ARM TrustZone and OP-TEE



Flow Summary



Only critical/sensitive CAN events are kept in the Secure World. Normal events are forwarded to the Normal World for execution.

BOOT PROCESS FLOW

Raspberry Pi 3 + ARM TrustZone + OP-TEE — Core1 Secure World

POWER ON
Raspberry Pi 3 SoC
↓
Boot ROM
↓
VideoCore
Reads first FAT32 boot partition

→

GPU BOOT
bootcode.bin
↓
start.elf + fixup*.dat
↓
config.txt
Loads boot files from SD card

→

ARMSTUB + FIP
armstub8.bin
↓
Padded TF-A BL1
+
fip.bin
ARM stub contains BL1 + FIP

→

TRUSTED FIRMWARE-A
BL1
↓
BL2
↓
BL31
EL3 Secure Monitor

- BL32 — OP-TEE OS — Secure World
- BL33 — U-Boot — Non-Secure

→

U-BOOT
bootcmd
↓
Load DTB + kernel8.img
↓
Set bootargs → booti
Non-Secure / Normal World

LINUX KERNEL BOOT

LINUX SMP BRING-UP

CPU0	CPU1	CPU2	CPU3
Linux	Secure World	Linux	Linux

CPU1 online request
↓
Intercepted in EL3
↓
opteed_enter_secure_server_if_core1()
↓
CPU1 → Secure World
↓
rpi3_can_server_loop()

CPU1 PERMANENTLY RESERVED FOR SECURE WORLD

Linux continues with CPU0, CPU2 and CPU3. Linux reports CPU1 offline.

NORMAL WORLD — LINUX
CPU0, CPU2, CPU3 — Linux Kernel + Userspace
Applications: spi_poc_test · secure_can_demo · tee-supplicant

SECURE WORLD — OP-TEE
CPU1 — rpi3_can_server_loop()
SECURE CAN PROCESSING
- CAN interrupt handling
- SPI0 CAN access
- Secure validation

CAN INTERRUPT PATH
MCP2518FD CAN Controller (SPI0) → GPIO25 / INT → BCM2837 Interrupt Routing → Secure FIQ → CPU1 / OP-TEE

FINAL RUNTIME STATE: CPU0 + CPU2 + CPU3 → Linux / Normal World. CPU1 → OP-TEE / Secure World → Secure CAN Processing.

AFTER BOOT

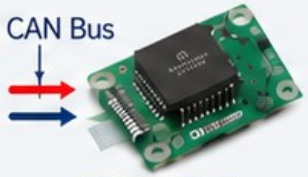
How a CAN Frame Is Processed

The secure CAN path runs on Core1 while Linux continues on the remaining cores.

SECURE WORLD — CORE1 (OP-TEE)

rpi3_can_server_loop() on Core1

CAN FRAME ARRIVES



MCP2518FD
CAN0 / SPI0

HARDWARE-PROVEN

GPIO25 / INT



HARDWARE-PROVEN

SECURE FIQ



HARDWARE-PROVEN

FRAME CAPTURE (in FIQ)

Read from MCP2518FD
→ Copy to secure buffer

HARDWARE-PROVEN

SECURE VALIDATION (in Core1 Secure Loop)

CAN ID check	HARDWARE-PROVEN
Counter / Replay check	HARDWARE-PROVEN
Timestamp check	HARDWARE-PROVEN
HMAC-SHA256	CURRENT BLOCKER

VALIDATION DECISION

Valid?

ACCEPT

DROP

SECURE LOGGING

NEXT STAGE

NORMAL WORLD — LINUX (CPU0 / CPU2 / CPU3)



Raspberry Pi 3 Model B

CPU0 Linux

CPU2 Linux

CPU3 Linux

Linux Kernel + Userspace

Linux continues normally on CPU0, CPU2 and CPU3.

HARDWARE-PROVEN

CURRENT POSITION

Core1 securely captures and validates CAN events;
HMAC authentication is the **current blocker** before secure logging.

Project Status & Roadmap

Three milestones hardware-proven, one blocked, three not yet started.



#	Pipeline Stage	Status	Evidence / Remarks
1	Boot & Core Pinning	HARDWARE-PROVEN	Core1 excluded from Linux SMP - confirmed via dmesg
2	Secure CAN Ownership	HARDWARE-PROVEN	SPI0 + GPIO25 owned by OP-TEE; CAN interrupt reaches Secure World as FIQ
3	CAN ID Check	HARDWARE-PROVEN	Single allowed ID 0x123, validated on Core1
4	Replay / Counter Check	HARDWARE-PROVEN	Validated on real captured frames
5	Timestamp Check	HARDWARE-PROVEN	Freshness-continuity tested only; no stale-frame rejection demonstrated
6	HMAC Authentication	BLOCKED	Unresolved Secure World runtime issue - hypothesis, not a diagnosis
7	Rate & Flood Detection	NOT STARTED	-
8	Tamper-Evident Logging	NOT STARTED	RPi3 has no RPMB; storage/integrity design still open

Next Steps

- Investigate and resolve HMAC runtime issue
- Design and implement rate & flood detection
- Implement tamper-evident logging with secure storage
- Complete final demo and collect metrics

What I'll Deliver Next

The concrete work planned between now and the final presentation.

HMAC authentication

Unresolved Secure World runtime issue - resolve or clearly document the root cause

Reduce validation latency

Measure and optimize latency / throughput / CPU-overhead of the validation pipeline

Store validated packets in flash memory

Tamper-evident logging design; RPi3 has no RPMB, storage boundary still to be designed

Rate & flood detection

Defend against a CAN-bus flood / denial-of-service

End-to-end demo with real measurements

Full hardware demonstration with real captured data, not placeholder values

By the final presentation: these five items resolved, implemented, or clearly documented - nothing promised beyond this list.

Thank You